sim_ex: Verifying a Concurrent DES Library with PropEr

Inter-Engine Equivalence, Causal Ordering, and the Bug 700 Sequences Found

Igor Ostaptchenko
io@octanix.com
Octanix Americas LLC / Wayne State University
Detroit, MI, USA

Abstract

Discrete event simulation engines on the BEAM have been demonstrated at scale—Sim-Diasca at EDF, InterSCSimulator for urban traffic—but their verification has been limited to point tests and trace-based assertions. We present a three-tier verification methodology for sim_ex, a multi-engine DES library for Elixir/OTP: point tests, property tests with a hybrid stochastic harness, and stateful command-sequence tests via `proper_statem`. Three `proper_statem` models generate approximately 700 adversarial command sequences per run in 1.5 seconds and found three bugs the existing suite had missed, including a spurious-release protocol error shrunk to four steps. We formulate inter-engine equivalence as a PropEr property and report results for strict and statistical equivalence across three of five backends. A probe against Sim-Diasca's scheduling core (1,600 trials, 8 properties, no failures) validates transferability. The methodology transfers; the postconditions are engine-specific.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; *Formal software verification*; • **Mathematics of computing** → Stochastic processes.

Keywords: property-based testing, discrete event simulation, Erlang, Elixir, BEAM, PropEr, proper_statem, inter-engine equivalence

ACM Reference Format:
Igor Ostaptchenko. 2026. sim_ex: Verifying a Concurrent DES Library with PropEr: Inter-Engine Equivalence, Causal Ordering, and the Bug 700 Sequences Found. In *Proceedings of 25th ACM SIGPLAN Erlang Workshop (Erlang '26)*. ACM, New York, NY, USA, 5 pages.

1 Introduction

Discrete event simulation on the BEAM is not new. Sim-Diasca ran million-actor simulations at EDF in 2010 [1]; InterSCSimulator scaled urban traffic models to comparable sizes [2].

What has received less attention is adversarial verification of the engine itself—the event calendar, the dispatch loop, the resource protocol, the causal ordering guarantees. We have not found published work that subjects DES engine internals to property-based or stateful randomized testing. Point tests catch the bugs the developer anticipated; the bugs that matter are the ones nobody did.

Property-based testing generates random inputs and checks invariants, with automatic shrinking to minimal counterexamples [3]. Stateful PBT via `proper_statem` [4] generates random *command sequences*, verifying postconditions after each step. Applied to Erlang/OTP at Ericsson and Volvo [5–7], but not, to our knowledge, to a discrete event simulation engine.

This paper presents the verification methodology for sim_ex, an open-source multi-engine DES library for Elixir/OTP with five backends, a 14-verb DSL, and a Rust NIF engine. Five contributions:

1. A **three-tier testing taxonomy**—*clipboard* (point tests), *auditor* (property tests with shrinking), *saboteur* (stateful command-sequence tests)—and an account of how the tiers complement each other.
2. A **hybrid stochastic harness** that separates deterministic invariants (shrinkable) from statistical invariants (not shrinkable) to prevent PropCheck's shrinker from chasing Monte Carlo noise.
3. Three **`proper_statem` models** generating ∼700 adversarial command sequences per test run in 1.5 seconds.
4. **Three bugs found**—a spurious-release protocol error (four-step shrunk sequence), a missing pattern clause in the parallel engine, and a silent integration failure between the DSL and the Rust NIF that returns `{:ok, _}` with zero events processed.
5. **Inter-engine equivalence as a PropEr property**, with empirical results for strict equivalence (engine vs. ETS, 30 trials, all identical) and statistical equivalence (engine vs. diasca, 60 trials, within documented tolerance). The equivalence

Erlang '26, August 28, 2026, Indianapolis, IN, USA
2026.

work surfaced an undocumented architectural taxonomy: the five backends split into a *time group* (float) and a *tick group* (integer with (T, D) timestamps).

To test transferability, we extracted the pure scheduling functions from Sim-Diasca's `class_TimeManager.erl` and ran 1,600 PropEr trials across 8 properties. All passed, as one would expect of code in current service at EDF since fifteen years.

sim_ex is open-source under Apache-2.0. `mix test` reproduces every result in this paper.

Joe Armstrong was a physicist. Physicists bring *informed priors*—conservation laws, causality, monotonicity. Armstrong at Ericsson could not let a phone ring before it was dialed; Boudeville at EDF whose day job forbids releasing energy before fission. Causality was not a feature they added; it was the thing they knew how to draw.

But the simulation models that run *on* the BEAM inherit no such instincts. In a simulation, conservation holds only if someone wrote a postcondition to check it. Property-based testing is the conservation law made executable. PropEr enforces what gravity enforces for free. Joe always advised: solve the hardest problem first. In a simulation engine, the hardest problem is correctness under inputs no well-formed model would ever produce—the spurious release, the dangling event, the calendar that silently reorders—because the developer cannot imagine what the developer cannot imagine. That is the problem this paper takes first. The physicist built the machine; we are building the physics.

2 sim_ex Architecture

sim_ex maps Averill Law's simulation methodology [8] to OTP. Same model, five backends, no code changes.

2.1 Entities and the Event Loop

The fundamental abstraction is the `Sim.Entity` behaviour:

```
@callback init(config :: map()) :: {:ok,
     state}
@callback handle_event(event, clock,
    state)
     :: {:ok, new_state, [new_events]}
@callback statistics(state) :: map()
```

An entity receives an event and a clock, updates its state, returns new events. Calendar: `:gb_trees` keyed by {`time, sequence_number`} for FIFO tie-breaking. Statistics: Welford's online algorithm, constant memory.

2.2 Five Execution Backends

Engine (default). Tail-recursive loop, single process, Map for entity states, `:gb_trees` calendar. 270K events/sec on PHOLD.

ETS. Same loop, entity states in ETS with write concurrency. A stepping stone toward parallelism.

Diasca. Implements Sim-Diasca's causal ordering [1]. Events carry (T, D) timestamps; tick $T{+}1$ begins only at quiescence. Tags: `:same_tick` $\rightarrow(T, D{+}1)$, `:tick` $\rightarrow(T', 0)$, `:delay` $\rightarrow(T{+}\delta, 0)$.

Parallel. Extends Diasca with concurrent dispatch: events at (T, D) are causally independent and can be partitioned across a persistent worker pool.

Rust NIF. Entire simulation in one NIF call: BinaryHeap calendar, Vec states, `rand` crate. 6.4M events/sec—25× the Elixir engine per core. Accepts DSL-compiled step lists only.

2.3 The Engine Taxonomy

A discovery from the equivalence work (Section 6): the five engines split into two groups by time representation.

Group	Engines	Clock	Stop
Time	Eng., ETS	float	stop_time
Tick	Diasca, Par., Rust	{tick,diasca}	stop_tick

The protocols are incompatible (Section 5). This taxonomy was not designed; PropEr surfaced it on the first generated input.

2.4 The DSL

A 14-verb language inspired by GPSS [9] and Arena [10], compiled at `mix compile` time to `Sim.Entity` modules:

```
defmodule Barbershop do
  use Sim.DSL
  model :barbershop do
    resource :barber, capacity: 1
    process :customer do
      arrive every: exponential(18.0)
      seize :barber
      hold exponential(16.0)
      release :barber
      depart
    end
  end
end
```

`Sim.Source` detects the clock type by pattern matching and emits the appropriate event tags. One keyword change switches a DSL model between `:engine`, `:ets`, and `:diasca`—the natural input for equivalence testing.

2.5 Design Principles

Three principles matter for verification: (1) **functional PRNG**—`:rand` state threaded through entities; same seed, same trajectory; CRN for free. (2) **Process = entity (optional)**—the `Sim.Entity` behaviour is agnostic to hosting model; same code, five backends. (3) **Zero runtime dependencies**—`mix test` reproduces every result in this paper.

3 The Three-Tier Testing Taxonomy

A simulation engine's inputs are stochastic; its outputs are distributions. `grants >= releases` is a conservation law, not a software requirement. An engine tested without postconditions may be computationally valid but physically meaningless.

Three strategies: *the clipboard*, *the auditor*, *the saboteur*.

3.1 The Clipboard (Point Tests)

Fixed seed, expected output. Over a hundred tests, under two seconds. Weakness: the developer must imagine the bug to write the test.

3.2 The Auditor (Property Tests)

PropCheck [11] generates random M/M/c configurations and checks invariants: flow conservation, determinism, cross-mode equivalence, DSL verb semantics. On failure, shrinking isolates the minimal counterexample. ∼100 seconds. Weakness: it tests *scenarios*, not *operation sequences*—it will never release a job that was never seized.

3.3 The Saboteur (Stateful Tests)

`proper_statem` [4] generates random command sequences, verifying postconditions after every step. It releases jobs never seized, steps engines never initialized. Three models, ∼700 sequences, 1.5 seconds. The Resource model found the bug the clipboard and auditor missed (Section 5).

3.4 The Hybrid Stochastic Harness

Little's Law holds in expectation, not on finite runs. Written as a PropCheck property, the shrinker chases noise—reducing run length until the sample is too small.

Solution: **deterministic invariants** use PropCheck with shrinking. **Stochastic invariants** use a hand-rolled harness—deterministic seeds, no shrinking, runs long enough for convergence. Shrink what you can; replay what you can't.

4 proper_statem Models

Three models, different engine layers. Engine state cannot be a PropEr symbolic variable; the workaround is the process dictionary. Optimistic `next_state` updates (guarded by `when is_number(clock)`) allow meaningful symbolic-phase command generation.

Model 1 (Sim.Statham). Full engine, barbershop scenario. Commands: `init`, `step`, `run_n`, `check`. Postconditions: calendar sortedness, target validity, flow conservation, clock monotonicity. 200 sequences, 365 ms.

Model 2 (Sim.Statham.Resource). Seize/release protocol in isolation. The `release` command is generated only for held job IDs—the constraint that surfaced the bug. Postconditions: capacity containment, grant-release balance. 300 sequences, 135 ms.

Model 3 (Sim.Statham.Adversarial). Preemptive resource with rush/normal order sources. Exercises the generation-counter logic that invalidates stale hold-complete events. 200 sequences, 150 ms.

5 Found Bugs

Three bugs, none affecting a production model, all latent under the existing test strategy.

Bug 1: spurious release. Four-step shrunk sequence:

```
init(cap=3,preemptive), seize(1), release(1), release(3)
```

Job 3 was never seized; the engine accepted the release silently. `releases` exceeded `grants`. Fix: a `Map.has_key?` guard. Adversarial testing defends against bugs that do not yet exist.

Bug 2: parallel engine crash. Barbershop with `mode: :parallel` crashed in `insert_diasca_events/5`—the DSL emits float-time three-tuples; the parallel engine matches only diasca-tagged events. No test had ever run a DSL model on the parallel engine. A naive bridge clause caused infinite diasca progression (two BEAM processes at 140% CPU, killed with `SIGKILL`); the fix requires entity-side tick awareness in the DSL compiler.

Bug 3: Rust NIF silent no-op. `mode: :rust` returns `{:ok, %{events: 0}}`. The DSL passes `:entities`; the Rust engine expects step lists. It silently ignores the unrecognized options. Worse than a crash: it passes any `{:ok, _}` smoke test.

The saboteur does not respect the developer's mental model of what the system is for. That is the source of its value.

6 Inter-Engine Equivalence

Same model, same seed, two engines, different statistics: at least one is wrong.

6.1 Equivalence Relation

Strict (within the time group): bitwise-identical output.

```
r_eng.events == r_ets.events and
  r_eng.stats == r_ets.stats
```

Statistical (across the time/tick boundary): integer counts within max(5, 5%); float statistics within 35% relative (rounding compounds across service stages). Snapshot fields (`in_progress`, `busy`) are excluded—they depend on the exact stop boundary.

6.2 T1: Within-Time-Group

Engine vs. ETS, random M/M/c scenarios ($\rho \in [0.10, 0.85)$). 30 trials, all identical, 1.5 seconds.

6.3 T2': Cross-Group DSL Equivalence

Engine vs. ETS vs. Diasca, two DSL models (Barbershop and SplitCombine), 30 seeds each.

Model	Engine ↔ ETS	Engine ↔ Diasca
Barbershop	All identical	mean_wait 2–5%
SplitCombine	All identical	mean_wait up to 25%

180 runs, 3.2 seconds, 0 failures. SplitCombine's divergence: integer-tick rounding compounding across three stages. A consequence, not a bug.

6.4 Discovery: The Engine Taxonomy

An early equivalence property included `:parallel`. PropEr's first input failed: `:parallel` ignores `stop_time` and operates on {`tick, diasca`}. The property was wrong about what the engines *are*, not what they *should* do. PBT surfaces architectural knowledge documented nowhere.

7 Performance

All benchmarks: 88-core Xeon E5-2699 v4, OTP 27, Elixir 1.18.3.

Event throughput. Elixir engine: 270K events/sec (PHOLD, 100 LPs). Rust NIF: 6.4M events/sec (barbershop). DSL complexity tax between simplest and most complex model: 1.6×.

Parallel replication—the number that matters:

Configuration	Wall time	vs SimPy
SimPy (sequential)	∼6,300 ms	1.0×
Elixir parallel (88 cores)	683 ms	9.4×
Rust NIF parallel (88 cores)	207 ms	30×

1,000 replications of a 200K-event model. The analysis Law describes as rarely performed [8] completes in 207 ms. Functional PRNG, no shared state, no GIL.

Verification overhead. Full suite in ∼110 seconds. Fast enough for CI.

8 Related Work and Conclusion

8.1 BEAM-Based Simulation

Sim-Diasca [1] is the foundational BEAM DES engine. sim_ex's contribution is not the engine but the verification.

We extracted two pure functions from Sim-Diasca's `class_TimeManager.erl` (8,289 lines). 1,600 PropEr trials, 8 properties, no failures. Partly explained by architecture: Sim-Diasca's WOOPER class hierarchy prevents certain bug classes by construction—actors cannot emit calendar events directly, forge timestamps, or use unrecognized event tags. We tested the guts; the architecture protects the surface.

Two design points: Sim-Diasca prevents bugs architecturally; sim_ex permits flexibility and relies on testing. The methodology is necessary for the second, informative for the first. InterSCSimulator [2] and Toscano et al. [12] would benefit from the same approach.

8.2 Property-Based Testing

Heritage: Claessen and Hughes [3], Papadakis and Sagonas [4], Arts et al. [5], Hughes [6, 7]. Our contribution: the domain (DES), the hybrid harness, the equivalence framing.

8.3 Conclusion

Three tiers, three bugs, one architectural taxonomy the developers did not know they had. The Sim-Diasca probe passed 1,600 trials, validating transferability. The postcondition encodes what the developer knows about the physical system and constrains the simulation to the region where physics lives. The physicist built the machine; we are building the physics.

sim_ex, the test suite, and the Sim-Diasca probe are open-source at https://github.com/borodark/sim_ex.

References

[1] Olivier Boudeville. *Sim-Diasca: Technical Manual of the Sim-Diasca Simulation Engine*. EDF R&D, 2010. https://github.com/Olivier-Boudeville-EDF/Sim-Diasca

[2] Eduardo F. Z. Santana, Nelson Lago, Fabio Kon, and Dejan S. Milojicic. InterSCSimulator: Large-scale traffic simulation in smart cities using Erlang. In *Proc. MABS (Multi-Agent Based Simulation)*, LNCS, pp. 211–227. Springer, 2018.

[3] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proc. ICFP*, pp. 268–279. ACM, 2000.

[4] Manolis Papadakis and Kostis Sagonas. A PropEr integration of types and function specifications with property-based testing. In *Proc. Erlang Workshop*, pp. 39–50. ACM, 2011.

[5] Thomas Arts, John Hughes, Joakim Johansson, and Ulf Wiger. Testing telecoms software with Quviq QuickCheck. In *Proc. Erlang Workshop*, pp. 2–10. ACM, 2006.

[6] John Hughes. QuickCheck testing for fun and profit. In *Proc. PADL*, LNCS 4354, pp. 1–32. Springer, 2007.

[7] John Hughes. Experiences with QuickCheck: Testing the hard stuff and staying sane. In *A List of Successes That Can Change the World*, LNCS 9600, pp. 169–186. Springer, 2016.

[8] Averill M. Law. *Simulation Modeling and Analysis*. McGraw-Hill, 5th edition, 2014.

[9] Geoffrey Gordon. A general purpose systems simulation program. In *Proc. EJCC*, pp. 87–104. Macmillan, 1961.

[10] W. David Kelton, Randall P. Sadowski, and Nancy B. Zupick. *Simulation with Arena*. McGraw-Hill, 6th edition, 2015.

[11] Klaus Alfert. PropCheck: Property-based testing for Elixir. https://github.com/alfert/propcheck, 2019.

[12] Luca Toscano, Gabriele D'Angelo, and Moreno Marzolla. Parallel discrete event simulation with Erlang. *arXiv:1206.2775*, 2012.